

```

17. # Set up the constants:
18. EMPTY_SPACE = '.'
19. GRID_LENGTH = 9
20. BOX_LENGTH = 3
21. FULL_GRID_SIZE = GRID_LENGTH * GRID_LENGTH
22.
23.
24. class SudokuGrid:
25.     def __init__(self, originalSetup):
26.         # originalSetup is a string of 81 characters for the puzzle
27.         # setup, with numbers and periods (for the blank spaces).
28.         # See https://inventwithpython.com/sudokupuzzles.txt
29.         self.originalSetup = originalSetup
30.
31.         # The state of the sudoku grid is represented by a dictionary
32.         # with (x, y) keys and values of the number (as a string) at
33.         # that space.
34.         self.grid = {}
35.         self.resetGrid() # Set the grid state to its original setup.
36.         self.moves = [] # Tracks each move for the undo feature.
37.
38.     def resetGrid(self):
39.         """Reset the state of the grid, tracked by self.grid, to the
40.         state in self.originalSetup."""
41.         for x in range(1, GRID_LENGTH + 1):
42.             for y in range(1, GRID_LENGTH + 1):
43.                 self.grid[(x, y)] = EMPTY_SPACE
44.
45.         assert len(self.originalSetup) == FULL_GRID_SIZE
46.         i = 0 # i goes from 0 to 80
47.         y = 0 # y goes from 0 to 8
48.         while i < FULL_GRID_SIZE:
49.             for x in range(GRID_LENGTH):
50.                 self.grid[(x, y)] = self.originalSetup[i]
51.                 i += 1
52.             y += 1
53.
54.     def makeMove(self, column, row, number):
55.         """Place the number at the column (a letter from A to I) and row
56.         (an integer from 1 to 9) on the grid."""
57.         x = 'ABCDEFGHI'.find(column) # Convert this to an integer.
58.         y = int(row) - 1
59.
60.         # Check if the move is being made on a "given" number:
61.         if self.originalSetup[y * GRID_LENGTH + x] != EMPTY_SPACE:
62.             return False
63.
64.         self.grid[(x, y)] = number # Place this number on the grid.
65.
66.         # We need to store a separate copy of the dictionary object:
67.         self.moves.append(copy.copy(self.grid))
68.         return True
69.
70.     def undo(self):
71.         """Set the current grid state to the previous state in the

```